

Intelligent University Course Finder

An AI-powered discovery system that helps students navigate university courses effectively. By combining Hybrid Retrieval (Vector + Keyword) with a pipeline of specialized AI Agents, the system understands messy human intents, identifies skill gaps, aligns courses to career tracks, and generates a personalized, difficulty-staged learning path.

■ Key Features

- **Agentic Pipeline (Parallel Processing):** Built with the OpenAI Agents SDK. Runs SkillGapAgent and CareerAgent in parallel, cutting logical latency in half.
 - **Hybrid Retrieval Framework:** Fuses Qdrant Dense Vector Search (semantic) with BM25 Keyword Search (exact token match) using Reciprocal Rank Fusion (RRF), ensuring no keyword or conceptual meaning is missed.
 - **Parent/Child Domain Chunking:** Courses are broken down into Identity, Skills, and Description "child" query variants, while the entire rich "parent" course object is returned to the agent without noisy embedding pollution.
 - **Zero-cost Deterministic Sequencer:** A stateless python-based module accurately sequence courses by difficulty levels without additional LLM API costs.
 - **Pre-Emptive Security Guardrails:**
 - **Presidio Analyzer / Anonymizer:** Detects and redacts PII elements client side *before* any prompt leaves the local machine.
 - **LangChain Semantic Intent Validation:** Rejects off-topic, toxic, or prompt-injection queries pre-emptively on the gateway.
 - **Modern Microservice Architecture:**
 - FastAPI **API Gateway** acting as an intermediate decoupling surface processing timeouts.
 - Dedicated **FastAPI Microservice** orchestrating AI Logic and Database calls.
 - **React + Vite** frontend implementing a clean, modular glassmorphism dark theme.
-

■■ Architecture

The system is separated into structured layers:

1. **Frontend (React):** Component-driven interface (Search, Career Card, Advisor Summary, Course Path).
2. **API Gateway (FastAPI):** Single proxy endpoint (Port 8080) for frontend isolation and traffic logging.
3. **Core API (FastAPI):** Central Microservice (Port 8000) orchestrating deterministically over the Agent workflows.

4. **Guardrails (Presidio & LangChain):** Pre-processing queries to remove PII and validate learning intent prior to LLM submission.
5. **Agent Pipeline (OpenAI SDK):**
 6. *IntentAgent:* Normalizes raw queries into structured {topic, level, keywords} filters.
 7. *SkillGapAgent:* Proactively highlights prerequisites missing from the student configuration.
 8. *CareerAgent:* Assigns courses to definitive job tracks.
 9. *LearningAdvisorAgent:* Summarizes and structures actionable human-friendly tips.
10. **Retrieval:** Qdrant Vector DB & BM25 Pickle Index.

■ For a file-by-file breakdown, refer to the **Artifacts/CODE_STRUCTURE.md** and the design rationale in **Artifacts/design_decisions.md**.

■ Getting Started

Prerequisites

- Python 3.10+
- Node.js (v18+) & NPM
- OpenAI API Key

1. Installation

Clone the repo, install python packages, and install UI packages:

```
# Backend python dependencies pip install -r requirements.txt # Download Spacy model for  
Presidio PII Detection python -m spacy download en_core_web_lg # Install frontend  
dependencies cd frontend/careerguidefrontend npm install cd ../../
```

2. Configuration

Create a `.env` file in the root directory:

```
OPENAI_API_KEY=sk-... LLM_MODEL=gpt-4o QDRANT_PATH=storage/qdrant_db  
BM25_INDEX_PATH=storage/bm25_index.pkl FRONTEND_URL=http://localhost:5173
```

3. Data Ingestion

You must initialize the Vector & Keyword databases before querying.

```
python ingestion/ingest.py
```

This will parse `data/coursera_courses.csv`, chunk documents natively into parent/child formats, embed them with OpenAI text-embedding-3-small, upsert to `storage/qdrant_db`, and pickle the `storage/bm25_index.pkl` keyword store.

■ Running the Application

To start the full environment, you will need **3 terminal tabs**.

Terminal 1: Start the Backend Microservice (Port 8000)

```
uvicorn api.main:app --port 8000 --reload
```

Terminal 2: Start the API Gateway (Port 8080)

```
uvicorn api_gateway.gateway:app --port 8080 --reload
```

Terminal 3: Start the UI / Frontend (Port 5173)

```
cd frontend/careerguidefrontend npm run dev
```

Finally, open **http://localhost:5173** in your browser.

■ Testing

Standalone test scripts are located in the `/test` directory. Use them to debug specific nodes in the graph in isolation.

- **Check Qdrant DB points/connections:** `python test/check_qdrant.py`
- **Test Hybrid Retriever & Filters:** `python test/test_retriever.py`
- **Test Guardrails (PII & Semantic Intent):** `python test/test_guardrails.py`
- **Test IntentAgent (Structured Extraction):** `python test/test_intent_agent.py`

■ Logging

The application utilizes centralized structured logging logic via `logger.py`. Logs are separated per-module in the `/logs` directory (e.g., `logs/guardrails.log`, `logs/intent_agent.log`) enabling granular debugging without polluting output.